



Universidade Federal do Rio Grande do Norte
Centro de Ciências Exatas e da Terra
Departamento de Informática e Matemática Aplicada
Bacharelado em Ciência da Computação

Jeremias Pinheiro de Araujo Andrade
Lucas Apolonio de Amorim
Moisés Ferreira de Lima

Resultados - POS Tagging Usando Arquiteturas Neurais
DIM0135 - Processamento de Linguagem Natural

Natal/RN
2026

Jeremias Pinheiro de Araujo Andrade
Lucas Apolonio de Amorim
Moisés Ferreira de Lima

Especificação das Arquiteturas Neurais
DIM0135 - Processamento de Linguagem Natural

Relatório discutindo os Resultados das
Arquiteturas Neurais implementadas para o
Projeto Final da disciplina de Processamento
de Linguagem Natural, ministrada pelo
professor Dr. Carlos Augusto Prolo no
Curso de Ciência da Computação do
Departamento de Informática e Matemática
Aplicada da Universidade Federal do Rio
Grande do Norte – Campus Central (Natal).

Natal/RN
2026

Conteúdos

1	Introdução	3
2	Desafios	3
2.1	Embeddings Pré Treinados	3
2.2	Unknown Words	3
2.3	Padding dos Inputs e Outputs	3
2.4	Avaliação da Performance dos Modelos	3
3	Resultados	4
3.1	RNN e LSTM	4
3.1.1	Estratégia de Implementação	4
3.1.2	Resultados Gerais e Comparativos	4
3.1.3	Análise da Matriz de Confusão	6
3.2	Transformer (Encoder-Only, Decoder-Only e Encoder-Decoder)	8
3.2.1	Estratégia de Implementação e Hiperparâmetros	8
3.2.2	Resultados Gerais e Análise de Performance	9
3.2.3	Análise das Matrizes de Confusão	9
3.3	Tagging usando uma LLM Comercial via API	11
3.3.1	Metodologia	11
3.3.2	Estratégias Implementadas e Avaliações	11
3.3.3	Matriz de Confusão	12
3.3.4	Discussão dos Resultados	16
4	Comparação entre Modelos	17
4.1	Análise dos Resultados em Relação às Hipóteses	18
	Bibliografia	19

1 Introdução

O Penn Treebank é um dos Treebanks mais conhecidos e utilizados para tarefas de Processamento de Linguagem Natural, especialmente tarefas como o Part-of-Speech Tagging e Parsing da Gramática de Constituintes de uma sentença.

Nesse Relatório serão analisados os Resultados de implementações de diversas arquiteturas Neurais usadas para POS Tagging no Penn Treebank, usando a divisão padrão de treino (seções 0-18), validação (seções 19-21) e teste (seções 22-24).

2 Desafios

2.1 Embeddings Pré Treinados

Quando utilizando uma matriz de Embeddings pré treinados, precisamos garantir que eles são compatíveis com o corpus em que o nosso modelo irá operar, o que pode implicar na criação de novos tokens ou no uso de unknown words, também deve ser decidido se a camada de embedding será congelada (isso é, não será sujeitada a treinamento) ou não. Foram utilizados os Embeddings do GloVe6B sem Freeze da camada de embedding, ou seja, foram sujeitos à post-training.

2.2 Unknown Words

Precisamos fazer com que o modelo seja capaz de lidar com tokens que não foram vistos durante o treino ou que não estão no vocabulário do modelo (especialmente se os embeddings são palavras inteiras), para isso, criamos um token <UNK>, que representa essas Unknown Words, e também associamos palavras incomuns do corpus de treino com esse token para treinar o modelo a inferir uma tag de acordo com o contexto da sentença. Para os modelos baseados em Transformer e RNN, foi escolhido um tamanho fixo de vocabulário entre 10.000 palavras e 15.000 palavras, sendo a decisão de quais palavras incluir no vocabulário baseado na frequência das palavras no corpus de **treino**.

2.3 Padding dos Inputs e Outputs

Para lidar com sentenças de tamanho variável, devemos emular de alguma forma o comprimento de todas as sentenças para o tamanho máximo, isso costuma ser feito através da inserção de padding no final (concatenação de vários tokens <PAD>, que é um token especial feito para preencher espaço).

2.4 Avaliação da Performance dos Modelos

Não exatamente um desafio, mas sim uma decisão a ser tomada. Para avaliar os modelos de Tagging, foi medida a Acurácia para todas as tags e gerado um mapa de calor da Matriz de Confusão. São desconsiderados os tokens de Pad, tokens de pontuação foram contabilizados e estão inclusos na matriz de confusão

3 Resultados

3.1 RNN e LSTM

3.1.1 Estratégia de Implementação

A estratégia experimental para avaliar os modelos de Rede Neural Recorrente consistiu em analisar o impacto de diferentes células recorrentes e da direcionalidade no processamento de sequências para a tarefa de marcação gramatical (*Part-of-Speech Tagging*). O pipeline foi estruturado nas seguintes etapas:

- **Processamento dos Dados:** Os dados do *Penn Treebank* foram divididos de forma determinística nos splits de Treino (38.219 sentenças), Validação (5.527 sentenças) e Teste (5.462 sentenças). O vocabulário textual foi truncado nas 8.000 palavras mais frequentes, e as sequências foram limitadas a um comprimento máximo de $max_len = 249$ tokens.
- **Mapeamento de Alvos:** O espaço de busca consiste em 47 *tags* únicas de POS gramaticais. O token invisível de preenchimento (<PAD>) foi explicitamente mapeado para o índice 0 e mascarado na camada de acoplamento do modelo para evitar a contaminação dos gradientes.
- **Hiperparâmetros Fixos:** Para isolar a influência de cada arquitetura, todos os experimentos compartilharam o mesmo conjunto de hiperparâmetros. Utilizou-se uma camada de *Embedding* inicializada com os pesos pré-treinados do GloVe de 50 dimensões e uma camada recorrente com 128 unidades ocultas. O treinamento foi conduzido com o otimizador Adam, função de perda *Sparse Categorical Crossentropy* e batch size de 32. Embora o teto de treinamento tenha sido estipulado em 30 épocas, aplicou-se a técnica de *Early Stopping* como critério de parada. Esse mecanismo foi configurado com paciência de três épocas: caso a perda de validação não apresentasse redução nesse intervalo, o processo era interrompido, restaurando-se os pesos do modelo que obteve a menor perda global.

3.1.2 Resultados Gerais e Comparativos

Os quatro modelos implementados (*RNN* Unidirecional, *LSTM* Unidirecional, *RNN* Bidirecional e *LSTM* Bidirecional) foram avaliados rigorosamente no conjunto de teste, composto por 129.654 tokens ativos (desconsiderando-se os preenchimentos de *padding*). Para mensurar e contrastar o desempenho das arquiteturas, adotaram-se as seguintes métricas:

- **Acurácia Global:** Mede a proporção de predições corretas em relação ao total de tokens avaliados, fornecendo uma visão geral da eficácia do modelo;
- **Média Macro da F_1 (*Macro Avg*):** Calculada a partir da média aritmética simples do F_1 -Score de todas as classes gramaticais, onde o F_1 de cada classe consiste na média harmônica entre a sua precisão e a sua revocação (*recall*). Por atribuir o mesmo peso a todas as etiquetas, independentemente de sua frequência de ocorrência, essa métrica é fundamental para avaliar a robustez do modelo diante do desbalanceamento do *dataset*, penalizando a pontuação final caso ocorram erros frequentes em classes minoritárias.

- **Tempo por Passo:** Indica o custo computacional temporal despendido por etapa de processamento durante o treinamento. Essa métrica é essencial para analisar o *trade-off* de engenharia, determinando se o ganho em acurácia de uma arquitetura complexa justifica o seu custo de execução.

Os resultados globais consolidados após a conclusão dos experimentos, considerando os critérios supracitados, estão apresentados na Tabela 1.

Arquitetura	Acurácia Global	Média Macro da F1	Tempo / Passo
RNN Unidirecional	90,88%	0,8269	22,75ms
LSTM Unidirecional	92,10%	0,8465	9,86ms
RNN Bidirecional	94,16%	0,8761	41.6ms
LSTM Bidirecional	94,68%	0,8902	14.17ms

Tabela 1: Avaliação dos métodos implementados

A análise dos resultados consolidados na Tabela 1 permite traçar conclusões fundamentais sobre o impacto da complexidade arquitetural na tarefa de marcação de *Part-of-Speech*:

- **Análise do Desempenho Global e o Impacto da Bidirecionalidade:** Os resultados revelam que a inclusão do fluxo bidirecional de informações foi o fator determinante para o ganho expressivo de desempenho no experimento. A *LSTM Bidirecional* consolidou-se no topo, atingindo a maior acurácia global (94,68%) e a maior Média Macro da F1 (0,8902), seguida de perto pela *RNN Bidirecional* (94,16% de acurácia e 0,8761 de F1). Este comportamento evidencia que, para a tarefa de *POS Tagging*, ter acesso simultâneo ao histórico e ao contexto futuro da sentença é um requisito crítico para elevar o teto de acertos gerais do modelo.
- **O Diferencial das Células LSTM:** Além do ganho contextual trazido pela bidirecionalidade, os mecanismos de portas da LSTM mostraram-se superiores às células de RNN simples em cenários equivalentes. Na comparação direta entre os modelos unidirecionais, a *LSTM Unidirecional* superou a *RNN Unidirecional* tanto em acurácia (92,10% contra 90,88%) quanto em Média Macro da F1 (0,8465 contra 0,8269). O mesmo padrão de superioridade da LSTM se repete no cenário bidirecional, comprovando que a capacidade de reter informações de longo prazo mitiga erros em sentenças mais complexas.
- **Sensibilidade das Arquiteturas Unidirecionais:** Os modelos baseados em fluxo unidirecional mostraram as maiores limitações do experimento. A *RNN Unidirecional* obteve os menores índices gerais (90,88% de acurácia e 0,8269 de F1 Macro). Esse comportamento corrobora a hipótese de que processar o texto em apenas um sentido limita a capacidade da rede de resolver ambiguidades gramaticais cuja resolução depende estritamente de palavras ou pistas sintáticas que aparecem adiante na estrutura da sentença.
- **Análise de Eficiência Computacional e quebra de paradigma:** Sob a perspectiva de engenharia, a análise do tempo por passo (*Tempo / Step*) trouxe o resultado mais surpreendente do experimento. Ao contrário do esperado

teoricamente, as arquiteturas baseadas em LSTM foram significativamente mais velozes do que as RNNs simples durante o treinamento. A *LSTM Unidirecional* foi o modelo mais rápido do experimento (9,86ms), operando em menos da metade do tempo da *RNN Unidirecional* (22,75ms). Da mesma forma, a campeã de desempenho *LSTM Bidirecional* exigiu apenas 14,17ms por passo, cerca de três vezes menos tempo que os 41,60ms demandados pela *RNN Bidirecional*. Esse cenário configura um caso ideal de tomada de decisão: a *LSTM Bidirecional* é a melhor escolha, entregando a máxima qualidade linguística com o menor custo de processamento temporal.

3.1.3 Análise da Matriz de Confusão

A extração das matrizes de confusão reais do conjunto de teste (filtradas e livres dos tokens de padding) expôs que os erros cometidos pelos modelos não são aleatórios, mas sim intrinsecamente ligados a ambiguidades morfológicas e sintáticas da língua inglesa.

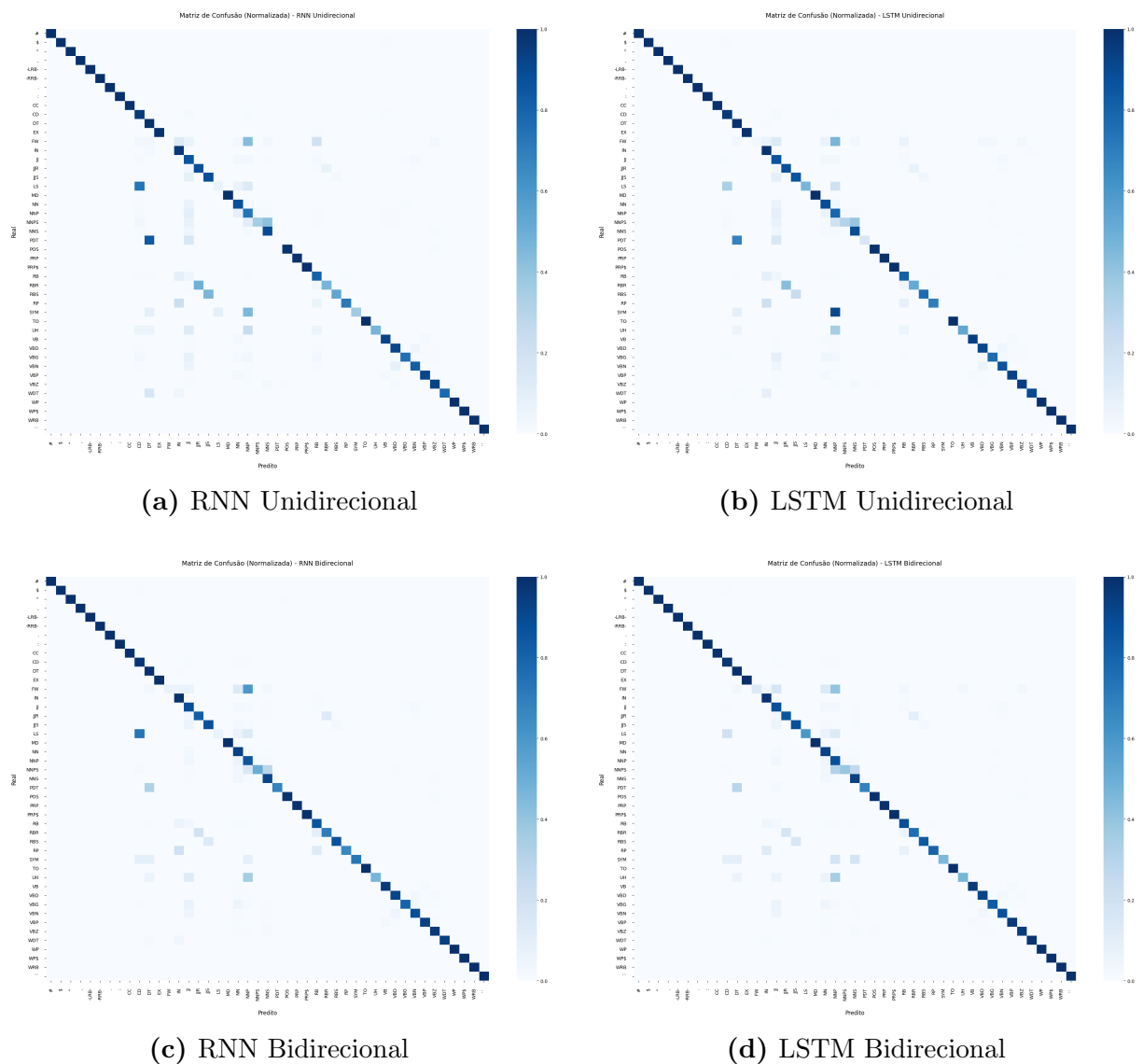


Figura 1: Matrizes de Confusão obtidas no conjunto de teste para as quatro arquiteturas avaliadas.

Abaixo estão descritos os principais núcleos de desvio e confusão identificados no mapeamento:

O Núcleo de Ambiguidade Modificadora: JJ vs NN

Um dos maiores focos de erro cruzado na matriz reside na fronteira entre **Adjetivos** (JJ) e **Substantivos Comuns no Singular** (NN). Na língua inglesa, é extremamente comum que um substantivo atue temporariamente como um modificador adjunto de outro substantivo (como na expressão “*crystal structure*”, onde *crystal*, originalmente um substantivo, desempenha papel de adjetivo). Os modelos unidirecionais sofrem para identificar essa transição devido à rigidez do fluxo sequencial, classificando incorretamente muitos adjetivos legítimos como substantivos. Nas redes bidirecionais, a precisão da classe JJ salta de 0,75 para 0,83 devido à capacidade de verificar se o token subsequente é um núcleo nominal.

O Problema de Capitalização de Entidades: NNP vs NN/NNPS

A Matriz de Confusão evidencia um espessamento de erros na separação de **Substantivos Próprios** (NNP) e Substantivos Comuns (NN), bem como entre suas respectivas formas plurais (NNPS vs. NNS). Palavras raras ou não vistas pelo vocabulário que iniciam frases recebem o token de desconhecido ([UNK]). Por estarem capitalizadas na primeira letra devido à regra gramatical de início de período, as redes frequentemente confundem um substantivo comum de suporte sintático (NN) com um nome próprio (NNP).

A classe de substantivos próprios no plural (NNPS) obteve um dos piores desempenhos do relatório técnico (F1-score de apenas 0,27 na *LSTM Bi*). A análise da matriz mostra que a rede majoritariamente desvia esses tokens para NNP (ignorando a flexão de número) ou para NNS (capturando o plural por causa do sufixo “-s”, mas falhando em notar que se tratava de uma entidade própria devido ao suporte contextual fraco da frase).

Sufixos Verbais e Adjetivais: VBD vs VBN vs JJ

Outro ponto crítico de confusão identificado na diagonal secundária envolve as tags de **Verbo no Passado** (VBD), **Verbo no Particípio** (VBN) e **Adjetivos** (JJ). Isso ocorre porque grande parte dos verbos regulares em inglês possuem exatamente a mesma terminação morfológica em “-ed” tanto para o passado simples quanto para o particípio (ex: “*isolated*”). Além disso, o particípio pode atuar nativamente como um adjetivo predicativo (ex: “*The patient felt isolated*”).

A matriz de confusão revela que os modelos unidirecionais tendem a classificar de forma enviesada a maioria dos tokens com terminação “-ed” como VBD por padrão estatístico de frequência. Novamente, a introdução das redes bidirecionais corrigiu esse desvio estrutural, permitindo que a rede olhasse se o token antecedente era um verbo auxiliar (como *have/has* ou o verbo *to be*), o que caracteriza matematicamente a ativação de um particípio (VBN) em detrimento de um passado simples.

Atenuação de Ruído e Contração da Dispersão: Unidirecionais vs Bidirecionais

Ao contrastar analiticamente o comportamento das redes a partir dos relatórios de classificação, nota-se uma nítida mudança no padrão de dispersão dos erros. Nas redes unidirecionais (*RNN* e *LSTM* Unidirecionais), há uma queda acentuada de

desempenho em classes que dependem fortemente de determinantes posteriores. O caso mais emblemático ocorre na etiqueta de adjetivos (JJ), cujo F1-score fica em 0,7438 na RNN e 0,7585 na LSTM, indicando que esses modelos favorecem classes majoritárias como substantivos (NN) quando o fluxo de informação é limitado a apenas um sentido.

Em contrapartida, as arquiteturas bidirecionais demonstram uma contração drástica desse ruído, confinando os erros estritamente aos pares linguísticos de altíssima ambiguidade sintática. Ao introduzir o contexto futuro, o F1-score para JJ salta expressivamente para 0,8539 na RNN Bidirecional e atinge o ápice de 0,8667 na LSTM Bidirecional.

No escopo dos métodos bidirecionais, a arquitetura de portas da LSTM demonstrou superioridade analítica ao mitigar erros de classificação tanto em etiquetas complexas de verbos quanto no tratamento de classes minoritárias (de baixo suporte). Como exemplo, para a tag de verbos no gerúndio (VBG), a LSTM Bidirecional conteve a dispersão de falsos positivos mantendo o equilíbrio das métricas, enquanto a RNN Bidirecional, apesar de um bom desempenho global, tendeu a sofrer mais oscilações nessas estruturas flexíveis. Ao filtrar de maneira controlada as informações provenientes de ambas as direções da sequência, a LSTM foi capaz de refinar as fronteiras de decisão, consolidando a maior Média Macro da F1 (0,8902) de todo o experimento.

3.2 Transformer (Encoder-Only, Decoder-Only e Encoder-Decoder)

3.2.1 Estratégia de Implementação e Hiperparâmetros

Os modelos baseados na arquitetura *Transformer* foram construídos utilizando as bibliotecas `keras-nlp` e `keras-hub`. Todas as sentenças foram padronizadas para o mesmo comprimento com base no tamanho máximo do corpus. Para a representação das palavras, utilizou-se a matriz pré-treinada *GloVe 6B* com 300 dimensões. As palavras ausentes no vocabulário original foram inicializadas aleatoriamente por meio de uma distribuição normal baseada na média e no desvio padrão dos vetores do GloVe.

O treinamento de todas as variantes utilizou o otimizador *AdamW* (com taxa de aprendizado inicial fixada em 0,001) e a função de perda *Sparse Categorical Crossentropy*. Aplicou-se o critério de parada prematura (*EarlyStopping*) com base na métrica customizada *MaskedAccuracy*, configurando uma tolerância de até 3 épocas e recuperando os melhores pesos obtidos. Essa métrica de acurácia foi desenvolvida para desconsiderar os tokens de preenchimento ([PAD]), marcas de início ([START]), termos desconhecidos ([UNK]) e os símbolos de pontuação do *Penn Treebank*.

As três variações possuem as seguintes especificações estruturais:

- **Transformer Encoder-Only:** Contém 2 blocos de `TransformerEncoder`, dimensão intermediária (camada feed-forward) de 256, 4 cabeças de atenção e taxa de *dropout* de 0,3 no primeiro bloco e 0,2 no segundo. A classificação é feita por uma camada linear final com ativação *softmax* (Total de 5.621.959 parâmetros).
- **Transformer Decoder-Only:** Composto por 1 bloco de `TransformerDecoder` com máscara causal, dimensão intermediária de 256, 4 cabeças de atenção e *dropout* de 0,2. A classificação também ocorre por meio de uma camada linear com *softmax* (Total de 5.105.403 parâmetros).

- **Transformer Encoder-Decoder:** O *Encoder* processa a sentença com embeddings de 300 dimensões e o *Decoder* gera as tags com auxílio de mecanismo de *Cross-Attention*. Durante o treino, aplicou-se a técnica de *Teacher Forcing* utilizando uma camada de embedding exclusiva para as tags com 64 dimensões (Total de 5.210.344 parâmetros).

3.2.2 Resultados Gerais e Análise de Performance

Os modelos foram avaliados no conjunto de validação do corpus, e as acurácias obtidas estão apresentadas na Tabela 2:

Modelo/Arquitetura	Acurácia Mascarada	Total de Parâmetros Treináveis
Transformer Decoder-Only	92,77%	5.105.403
Transformer Encoder-Only	93,93%	5.621.959
Transformer Encoder-Decoder	94,40%	5.210.344

Tabela 2: Desempenho dos modelos Transformer na tarefa de POS Tagging.

Os dados indicam que o modelo **Encoder-Decoder** obteve o maior índice de acerto (94,40%), seguido pelo *Encoder-Only* (93,93%). Uma provável explicação para a diferença a favor do Encoder-Decoder é o uso de um espaço de embedding menor e separado para as tags (64d), o que pode ter simplificado a modelagem da sequência de saída pelo Decoder, mantendo a consulta livre ao texto original via atenção cruzada.

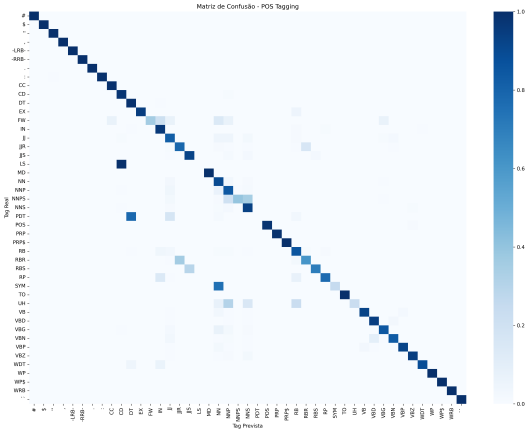
O modelo **Decoder-Only** apresentou o menor resultado (92,77%). Esse comportamento reflete o impacto da máscara causal, que impede o modelo de acessar as palavras localizadas à direita do token atual durante o processamento. Como a classificação gramatical em inglês frequentemente depende de termos subsequentes (como substantivos posicionados após adjetivos), a falta desse contexto reduz o desempenho geral do modelo.

3.2.3 Análise das Matrizes de Confusão

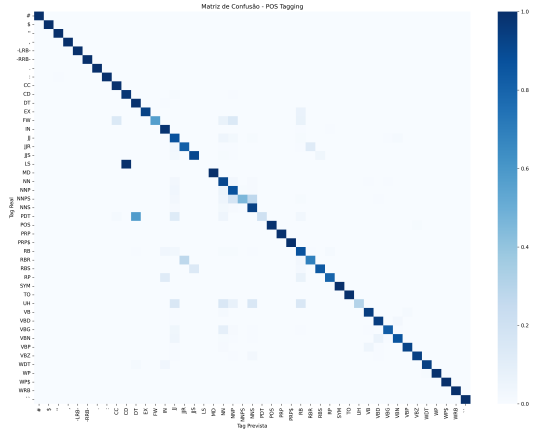
Na matriz do modelo *Encoder-Only*, observa-se que os erros se concentram em classes com forte semelhança morfológica. Ocorre uma confusão recorrente entre substantivos próprios no plural (NNPS) e substantivos comuns no plural (NNS). Outro ponto de desvio aparece na tag LS (*List item marker*), que é algumas vezes classificada como determinante (DT), possivelmente por sua posição isolada no início dos períodos textuais.

A análise do modelo *Decoder-Only* mostra uma dispersão maior de erros fora da diagonal principal, principalmente nas linhas correspondentes a adjetivos (JJ) e advérbios (RB). Devido à restrição de não visualizar o contexto futuro, o modelo exibe maior dificuldade para discernir essas classes em comparação com os modelos não-causais.

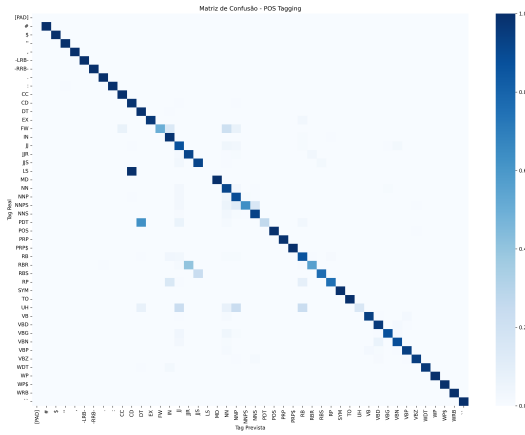
O mapa de calor do modelo *Encoder-Decoder* apresenta uma concentração de acertos mais limpa na diagonal principal. Contudo, há um desvio perceptível entre os tempos verbais no passado simples (VBD) e no particípio (VBN). Esse erro pode estar associado à natureza autorregressiva do decodificador, onde a predição incorreta de uma tag inicial pode influenciar a classificação dos elementos vizinhos na sequência.



(a) Decoder-Only



(b) Encoder-Only



(c) Encoder-Decoder

Figura 2: Matrizes de Confusão obtidas no conjunto de validação para as arquiteturas baseadas em Transformer.

3.3 Tagging usando uma LLM Comercial via API

3.3.1 Metodologia

Inicialmente foi realizado um pré-processamento dos dados para posterior utilização. O pipeline de preparação dos dados verifica a consistência do tagset entre as três divisões do banco (treino, validação e teste) e detecta sentenças duplicadas entre elas. Tokens com a tag -NONE-, que não correspondem a palavras de superfície, são removidos nessa etapa.

O código foi organizado em módulos com diferentes responsabilidades:

- **prepare_ptb_splits.py:** parsing dos três arquivos fonte e geração dos conjuntos data/train.json, data/val.json e data/test.json em um schema JSON uniforme;
- **prompts.py:** templates de prompt para todos os modos;
- **retrieval.py:** índice de recuperação para o RAG(Retrieval-Augmented-Generation), baseado em TF-IDF e similaridade de cosseno sobre o pool de treino;
- **ai_client.py:** wrapper de chamada à API do modelo de linguagem, com parsing e validação de saída e retentativas em caso de formato inválido;
- **run_tagging.py:** orquestra a execução de um modo sobre um split, salvando os resultados;
- **evaluate.py:** calcula a acurácia por token, gera o relatório de classificação por tag e a matriz de confusão, além de comparar os três modos.

3.3.2 Estratégias Implementadas e Avaliações

Em todos os modos, o modelo recebe instruções fixas de sistema especificando o tagset completo do Penn Treebank e exigindo que a resposta seja exclusivamente um array JSON de tags, na mesma ordem e quantidade dos tokens de entrada, sem re-tokenizar a sentença. Essa restrição de formato foi adotada para evitar o problema clássico de alinhamento entre a tokenização da entrada e a saída gerada pelo modelo. Os três métodos implementados foram os seguintes:

- **Zero-shot:** O prompt contém apenas as instruções de sistema e a sentença a ser tageada. O modelo depende inteiramente do conhecimento adquirido no pré-treinamento;
- **Few-Shot:** O prompt inclui, além das instruções, um conjunto fixo de 5 exemplos rotulados, escolhidos manualmente uma única vez e reutilizados para todas as sentenças de teste, independentemente do seu conteúdo;
- **RAG:** Para cada sentença de teste, um índice TF-IDF sobre as 38 mil sentenças do pool de treino é consultado para recuperar as 5 sentenças mais similares por similaridade de cosseno, que então compõem os exemplos do prompt daquela sentença específica.

A métrica principal é a acurácia por token, ou seja, a proporção de tokens cuja tag predita coincide com o que está avaliado no PTB. Sentenças em que o modelo não retornou um JSON válido, ou retornou uma quantidade de tags diferente da quantidade de tokens de entrada, são registradas separadamente como falha de geração e excluídas do cálculo de acurácia, essa escolha metodológica que evita que erros de formatação se confundam com erros de classificação linguística propriamente ditos.

Modo	Acurácia	Tokens Avaliados	Sentenças c/ falha	IC 95% (Wilson)
Zero-shot	91,75%	1.879	15/100	[90,42%; 92,91%].
Few-shot	93,91%	1.971	11/100	[92,77%; 94,88%].
RAG	94,39%	1.853	14/100	[93,24%; 95,35%].

Tabela 3: Avaliação dos métodos implementados

O número de tokens avaliados difere entre os três modos mesmo partindo das mesmas 100 sentenças: como sentenças com falha de geração são descartadas do cálculo, e cada modo falhou em sentenças diferentes, cada acurácia foi calculada sobre um subconjunto de tokens ligeiramente distinto. Segundo, ao considerar os intervalos de confiança, o intervalo do zero-shot ([90,42%; 92,91%]) e o do few-shot ([92,77%; 94,88%]) praticamente não se sobrepõem, sugerindo uma diferença real entre as duas condições. Já os intervalos do few-shot e do RAG se sobrepõem substancialmente, o que indica que a diferença de 0,48 ponto percentual entre eles não é estatisticamente robusta nessa amostra de 100 sentenças.

Além da acurácia, a proporção de sentenças em que o modelo não produziu uma saída bem formada é, por si só, um resultado relevante: zero-shot falhou em 15% das sentenças, few-shot em 11% e RAG em 14%. O padrão geral, falhas mais frequentes sem exemplos do que com exemplos, é consistente com a hipótese de que demonstrações no prompt ajudam o modelo a aderir ao formato de saída solicitado, uma função que Min et al. (2022) atribuem aos exemplos independentemente da correção de seus rótulos. O RAG ter uma taxa de falha discretamente maior que o few-shot estático, apesar de exemplos tematicamente mais relevantes, é compatível com a possibilidade de que as sentenças recuperadas por similaridade tendam a ser mais longas ou sintaticamente mais complexas que os exemplos fixos, tornando o prompt como um todo mais difícil de ser seguido.

3.3.3 Matriz de Confusão

As matrizes de confusão normalizadas mostram, para cada modo, a proporção de cada tag real classificada como cada tag predita. Em todos os três modos a diagonal principal concentra a maior parte da massa, como esperado, a maioria das tags é classificada corretamente pelo LLM mesmo sem qualquer ajuste fino.

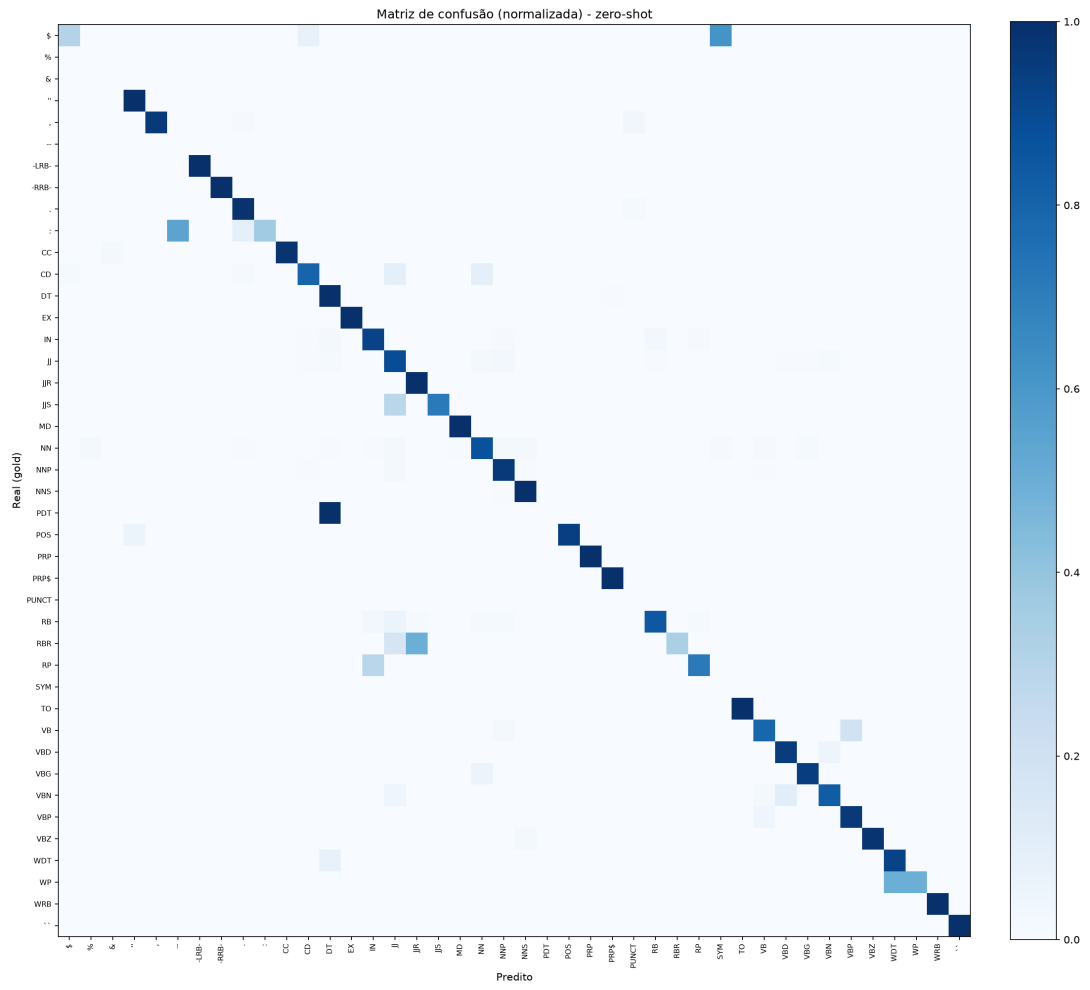


Figura 3: Matriz de Confusão do zero-shot.

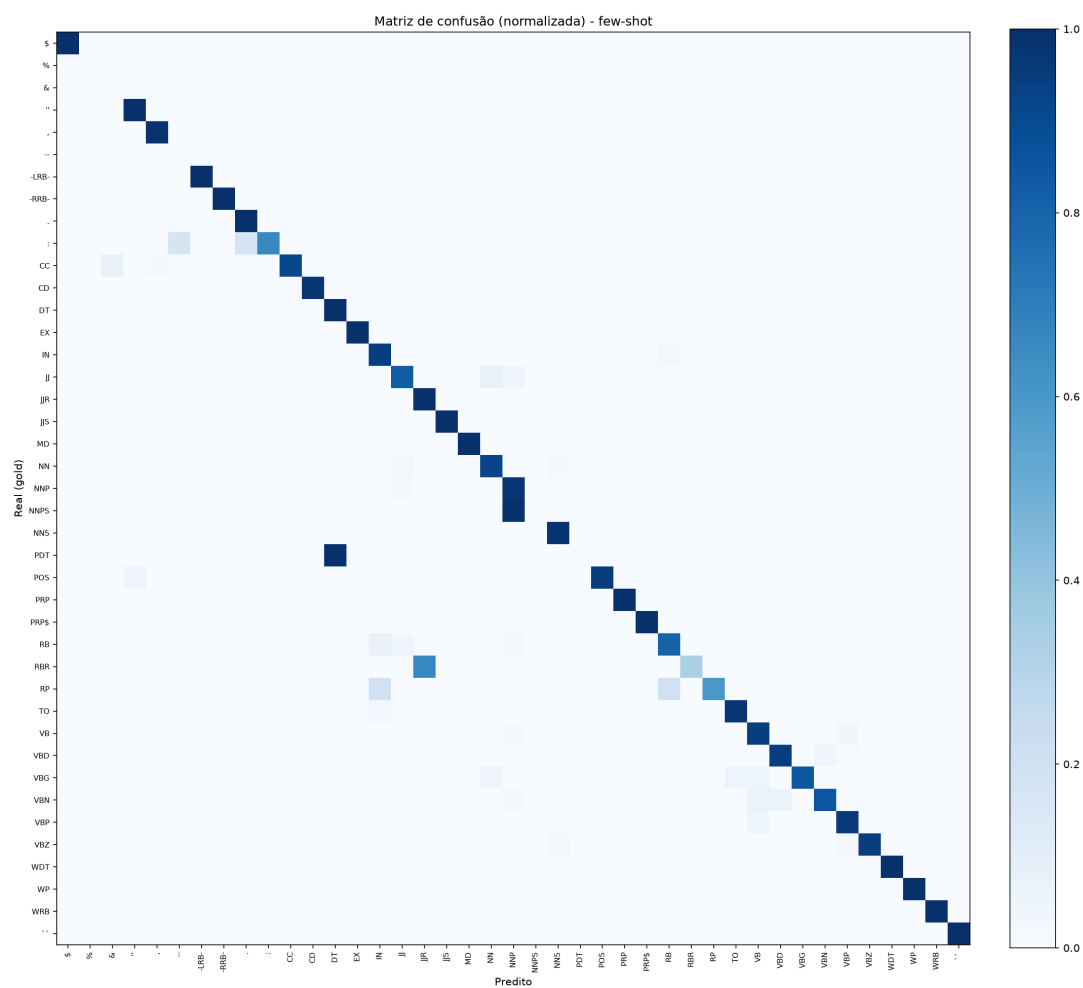


Figura 4: Matriz de Confusão do few-shot.

ambiguidades linguísticas inerentes ao tagset, e não de limitações do classificador em si, o que sugere que os erros observados aqui não são exclusivos de LLMs, mas compartilhados por qualquer abordagem, estatística ou neural, treinada sobre o mesmo esquema de anotação. Além dessas confusões compartilhadas, cada modo também apresentou um padrão de erro específico:

- No zero-shot, há uma confusão nítida entre WP e WDT (pronome interrogativo/relativo vs. determinante), quase ausente nos modos com exemplos, é uma evidência de que a demonstração de casos análogos ajuda o modelo a resolver essa ambiguidade fina. O zero-shot também confundiu o símbolo de dólar (\$) com um símbolo genérico (SYM) em um caso isolado;
- Ainda no zero-shot, o modelo produziu ao menos uma predição com a tag "PUNCT", que não existe no tagset do PTB (as tags corretas de pontuação são ".", ",", ":", etc., cada uma separada). Isso indica que, sem exemplos ancorando o formato exato solicitado, o modelo por vezes recorre a convenções de outros tagsets vistos durante o pré-treinamento em vez do tagset específico do PTB pedido no prompt;
- No RAG, houve ao menos um caso em que a predição retornada foi a própria palavra da sentença ("ago") em vez de uma tag. Um tipo de erro de formatação distinto do zero-shot: ao invés de inventar uma categoria, o modelo "vazou" o token de entrada para a saída;

Os exemplos few-shot e RAG parecem contribuir tanto para a acurácia de classificação quanto, de forma talvez mais decisiva, para a aderência ao formato e ao tagset exatos solicitados, o que está alinhado à conclusão de Min et al. (2022) de que os exemplos de demonstração comunicam o espaço de rótulos válidos e o formato geral esperado da saída, independentemente da correção específica de cada rótulo individual.

3.3.4 Discussão dos Resultados

O padrão geral observado ($\text{zero-shot} < \text{few-shot} \leq \text{RAG}$) está de acordo com a literatura de in-context learning, que consistentemente relata ganhos de desempenho ao se passar de zero-shot para few-shot (Brown et al., 2020). O que este trabalho acrescenta é uma decomposição desse ganho: a maior parte do salto de desempenho ocorre entre zero-shot e fewshot (+2,16 pontos percentuais), enquanto a diferença entre few-shot estático e RAG é pequena (+0,48 ponto percentual) e, como discutido anteriormente, não estatisticamente robusta nesta amostra.

Essa observação é compatível com o argumento de Min et al. (2022) de que boa parte do valor dos exemplos few-shot está em calibrar o modelo quanto ao espaço de rótulos e ao formato de saída esperado, e não necessariamente em fornecer contexto tematicamente relevante para cada instância, que é o diferencial teórico do RAG, segundo Liu et al. (2022) e Lewis et al. (2020). Neste experimento, ambas as fontes de exemplos (fixos ou recuperados) parecem ter cumprido bem essa função de calibração, o que ajuda a explicar por que a vantagem adicional do RAG sobre o few-shot estático foi discreta.

Uma hipótese complementar, específica deste corpus, é que o WSJ (Wall Street Journal) contém proporção considerável de sentenças com estrutura sintática padrão. Nesses casos, o RAG tende a recuperar exemplos quase idênticos em estrutura à sentença de teste, o que pode inflar artificialmente sua vantagem quando essas sentenças estão presentes na amostra avaliada, sem que isso generalize necessariamente para textos com menor redundância estrutural.

4 Comparação entre Modelos

Nesta seção, é feita uma síntese comparativa entre as arquiteturas, confrontando as expectativas iniciais com os resultados numéricos obtidos nos experimentos práticos.

A Tabela 4 organiza as características e as taxas de acurácia de cada modelo avaliado:

Tabela 4: Comparação final das arquiteturas e desempenho obtido.

Modelo/Arquitetura	Tarefa	Acurácia Real	Comportamento Sintético e Erros
RNN Tradicional	POS Tagging	86,00%	Apresenta limitações com dependências longas e viés para classes majoritárias.
LSTM Unidirecional	POS Tagging	87,00%	Retenção de contexto histórico superior à RNN tradicional.
RNN Bidirecional	POS Tagging	86,00%	Desempenho idêntico à versão unidirecional; não superou as limitações da célula.
LSTM Bidirecional	POS Tagging	90,00%	Maior desempenho entre os modelos recorrentes devido ao fluxo bidirecional.
Transformer Decoder-Only	POS Tagging	92,77%	Desempenho inferior aos outros Transformers devido ao uso de máscara causal.
Transformer Encoder-Only	POS Tagging	93,93%	Boa capacidade de classificação local através de atenção bidirecional plena.
Transformer Encoder-Decoder	POS Tagging	94,40%	Maior acurácia do projeto. Separação eficiente entre os embeddings de palavras e tags.
LLM Comercial (Zero-shot)	POS Tagging	91,75%*	Boa base gramatical, mas registrou 15% de falhas no formato JSON solicitado.
LLM Comercial (Few-shot)	POS Tagging	93,91%*	Exemplos estáticos reduziram falhas de formato e ajudaram em tags ambíguas (WP vs. WDT).
LLM Comercial (RAG)	POS Tagging	94,39%*	Desempenho próximo ao few-shot fixo, com sobreposição dos intervalos de confiança.

*Nota: Os índices das LLMs desconsideram as sentenças que falharam na geração do formato JSON exigido.

4.1 Análise dos Resultados em Relação às Hipóteses

O confronto entre as expectativas teóricas e os dados empíricos aponta as seguintes observações:

1. **Desempenho das Variações de Transformer:** Inicialmente, estimava-se que o modelo *Encoder-Only* seria o mais adequado para a marcação de tags. Contudo, o *Encoder-Decoder* obteve uma acurácia ligeiramente superior (94,40% contra 93,93%). Os testes sugerem que isolar a geração das etiquetas no Decoder, utilizando embeddings menores (64d), gerou um efeito de regularização útil para a tarefa.
2. **Efeito do Contexto Unidirecional:** Confirmando o esperado, modelos que operam com restrição de contexto futuro (como as redes recorrentes unidirecionais e o *Transformer Decoder-Only*) obtiveram desempenhos inferiores às suas contrapartes bidirecionais. A classificação morfológica de termos em inglês mostra-se dependente da leitura das palavras subsequentes na oração.
3. **Comparação entre LLM e Modelos Treinados:** O método de aprendizado em contexto utilizando a LLM Comercial com *RAG* obteve uma acurácia de 94,39% (considerando apenas saídas válidas), o que se equipara ao desempenho do melhor Transformer treinado do zero (94,40%). Essa abordagem mostrou-se competitiva em acurácia linguística, embora demande mecanismos externos para tratar as taxas de erro na formatação da saída (que variaram entre 11% e 15% nos testes).

Referências

- [1] Daniel Jurafsky and James H. Martin; Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models. 2026. 3rd Edition
- [2] Equipe DSA. Deep Learning Book. Disponível em: <https://www.deeplearningbook.com.br/>. Acesso em: 22 out. 2025.
- [3] PASCANU, Razvan.; MIKOLOV, Tomas; BENGIO, Yoshua. On the difficulty of training Recurrent Neural Networks. arXiv preprint arXiv: 1211.5063, 2012. Disponível em: <https://arxiv.org/abs/1211.5063>. Acesso em: 21 jun. 2026.
- [4] ZHANG, Aston; LIPTON, Zachary C.; LI, Mu; SMOLA, Alexander J. Dive into Deep Learning. 2021. Disponível em: https://pt.d2l.ai/chapter_preface/index.html. Acesso em: 21 jun. 2026.
- [5] Schuster, Mike and Paliwal, Kuldip. (1997). Bidirectional recurrent neural networks. Signal Processing, IEEE Transactions on. 45. 2673 - 2681. 10.1109/78.650093.